

AI Software Engineering Agent

Full Project Roadmap & Build Plan

A platform where a GitHub issue goes in and a tested Pull Request comes out. Developer just reviews and merges.

"At ParaMai I manually copied GitHub tickets into Claude, got a fix, then applied it myself. This project automates that entire loop — ticket goes in, tested PR comes out, developer just approves."

CORE FLOW

1	GitHub issue created	QA creates a ticket in GitHub repo
2	Your UI fetches it	Spring Boot calls GitHub API automatically
3	QA assigns to Developer	Developer sees ticket in dashboard
4	Analyze with AI	Developer clicks one button
5	AI finds relevant code	RAG searches pgvector embeddings
6	AI writes the fix	Coding agent edits files on new branch
7	AI runs mvn test	Self-corrects up to 3 times if tests fail
8	PR created on GitHub	Developer reviews diff and merges

ROLES — 2 ONLY

QA	Create ticket · Assign to developer · Track ticket status
Developer	See assigned tickets · Analyze with AI · Approve plan · Review diff · Create PR

TECH STACK

Frontend	React + Tailwind	You know it

Backend	Java + Spring Boot + JWT	You know it
AI Service	Python + FastAPI	You know it
LLM	Gemini Flash	Free, you already know it
Agent Framework	LangChain + LangGraph	You know LangChain
Database	PostgreSQL + pgvector	One DB for everything
Integration	GitHub API	Core of the project
Language scope	Java Spring Boot only	Start simple, expand later

ARCHITECTURE & COMMUNICATION

React	Spring Boot	HTTP REST	User actions, repo name
Spring Boot	GitHub API	HTTP + Token	Repo name, fetch issues
GitHub API	Spring Boot	JSON response	Issues list
Spring Boot	Python Service	HTTP REST	Issue details, repo URL
Python	Gemini	LangChain API	Issue + code as prompt
Gemini	Python	Text response	Fixed code
Python	pgvector	SQL query	Embeddings, code chunks
Python	GitHub API	HTTP REST	Branch, commit, PR
Spring Boot	React	WebSocket/SE	Live agent progress

UI — 4 SCREENS ONLY

1. Dashboard	Ticket list + repo selector + basic stats
2. Ticket detail	Issue info + AI analysis + approve plan button

3. Agent live view	Step by step progress stream while AI is working
4. Diff + PR	Code diff viewer + create PR button

BUILD PHASES

Phase 1	Week 1 — Foundation Spring Boot setup · PostgreSQL · JWT login · 2 roles · GitHub API connect · Read issues in UI · React ticket list
Phase 2	Week 2 — RAG System Clone repo · Parse Java files · Split into chunks · Gemini embeddings · Store in pgvector · Semantic search
Phase 3	Week 3 — AI Agents FastAPI setup · Issue analyzer agent · Retrieval agent · Planner agent · Dev approves plan · Coding agent · Git tools
Phase 4	Week 4 — Testing + PR Run mvn test · Self-correction loop (max 3x) · Code diff in UI · Dev reviews diff · Create GitHub PR · Live WebSocket updates
Phase 5	After MVP — Polish ML priority prediction · Evaluation metrics · Token cost tracking · Docker sandbox · Add Python language support

TICKET LIFECYCLE

OPEN	ANALYZING	PLAN READY	PLAN APPROVED	IMPLEMENTING	TESTING	PR CREATED	RESOLVED
------	-----------	------------	---------------	--------------	---------	------------	----------

SELF CORRECTION LOOP

AI writes fix	Coding agent edits files on a new git branch
Run mvn test	Maven runs all *Test.java files automatically
Tests pass?	Yes → Create PR immediately
Tests fail?	AI reads error message — understands what went wrong
Retry (attempt 2)	AI fixes code again and runs tests
Retry (attempt 3)	One final attempt if still failing
Still failing?	Notify developer — human intervention needed

HOW TESTING WORKS

Every Java project has test files written by developers:

UserServiceTest.java – the answer key for UserService.java

A test checks 3 things:

- GIVEN – set up the situation
- WHEN – call the function
- THEN – check if result is correct

Your AI agent simply runs one command:

mvn test

Maven finds all *Test.java files, runs every @Test function, and reports results. Your AI reads the output — pass means create PR, fail means try again.

DUMMY TEST REPO — USER-SERVICE

Since you have no production code, we create a dummy Java Spring Boot project with intentional bugs. You control the bugs so you can verify the AI fixes them correctly.

#1	Null pointer exception	No null check when user email not found
#2	Wrong HTTP status code	Returns 200 instead of 404
#3	Missing input validation	Accepts empty or null email
#4	Wrong calculation logic	Incorrect price calculation
#5	Missing exception handling	No try-catch around database call

MVP CHECKLIST — BUILD THIS FIRST

[]	Login + 2 roles working
[]	Connect a GitHub repo
[]	Read GitHub issues in your UI
[]	Index repo code into pgvector

[]	AI finds relevant files for a ticket
[]	AI proposes fix plan, dev approves
[]	AI edits code on a new branch
[]	Run mvn test, retry if failed (max 3x)
[]	Show code diff to developer
[]	Developer clicks, GitHub PR created

AGENTIC AI — SIMPLE EXPLANATION

Normal AI — you ask one question, you get one answer. Done.

Agentic AI — you give a goal, it breaks into steps, uses tools, makes decisions, completes the goal on its own.

Your agent has these tools:

read_file('LoginService.java')	- reads a file
edit_file('LoginService.java')	- edits a file
run_tests()	- runs mvn test
create_branch('fix/issue-101')	- creates git branch
search_code('findUser')	- searches codebase

LangGraph defines the workflow map. The AI decides which tool to use at each step. You define the map — AI walks through it.

FOLDER STRUCTURE

frontend/	React app — 4 screens
backend/auth/	Login, JWT, roles
backend/github/	GitHub API calls
backend/issue/	Ticket management
backend/agent/	Talks to Python service
ai-service/agents/	All AI agents
ai-service/tools/	File tools, git tools, GitHub tools

ai-service/rag/	Code search system
database/	SQL files, pgvector setup

NEXT STEPS — IN ORDER

Step 1	Create dummy user-service Java repo on GitHub with 5 intentional bugs
Step 2	Create 5 GitHub issues for those bugs
Step 3	Start Phase 1 — Spring Boot setup and login
Step 4	Build piece by piece, understand every line of code
Step 5	Test each agent against one bug at a time

Java Spring Boot only · 2 roles · 4 screens · Build MVP first · Explain every line as we go